



KANDIDAT

10686

PRØVE

TDT4160 1 Datamaskiner

Emnekode	TDT4160
Vurderingsform	Skriftlig eksamen
Starttid	11.12.2025 15:00
Sluttid	11.12.2025 19:00
Sensurfrist	08.01.2026 23:59
PDF opprettet	02.10.2026 16:13

Forside

Oppgave	Tittel	Oppgavetype
i	Forside	Informasjon eller ressurser

Instruksjonssett

Oppgave	Tittel	Oppgavetype
1.1	Instruksjonssett	Fyll inn tekst

Enkeltsykelprosessor

Oppgave	Tittel	Oppgavetype
2.1	Kontrollsignaler 1	Fyll inn tekst
2.2	Forklare kontrollsignaler 1	Langsvar
2.3	Kontrollsignaler 2	Fyll inn tekst
2.4	Forklare kontrollsignaler 2	Langsvar

Aritmetisk-logisk enhet

Oppgave	Tittel	Oppgavetype
3.1	ALU kontrollsignaler	Fyll inn tekst
3.2	Forklare ALU kontrollsignaler	Langsvar

Multiplikator

Oppgave	Tittel	Oppgavetype
4.1	Verdier i produktregisteret	Fyll inn tekst

4.2	Forklare multiplikatoren	Langsvar
-----	--------------------------	----------

Flyttall

Oppgave	Tittel	Oppgavetype
---------	--------	-------------

5.1	Flyttall til desimaltall	Fyll inn tall
-----	--------------------------	---------------

5.2	Forklare flyttall	Langsvar
-----	-------------------	----------

Samlebånd

Oppgave	Tittel	Oppgavetype
---------	--------	-------------

6.1	Samlebånd, kontrollsignaler	Fyll inn tekst
-----	-----------------------------	----------------

6.2	Forklare samlebånd	Langsvar
-----	--------------------	----------

6.3	Korrekt utføring	Langsvar
-----	------------------	----------

Hurtigbuffer og virtuelt minne

Oppgave	Tittel	Oppgavetype
---------	--------	-------------

7.1	Adresseformat	Fyll inn tekst
-----	---------------	----------------

7.2	Forklare adresseformat	Langsvar
-----	------------------------	----------

7.3	Aksesstid	Fyll inn tall
-----	-----------	---------------

7.4	Forklare aksesstid	Langsvar
-----	--------------------	----------

7.5	TLB	Langsvar
-----	-----	----------

Prossessorer med høyere ytelse og parallelle datamaskiner

Oppgave	Tittel	Oppgavetype
---------	--------	-------------

8.1	Register renaming	Langsvar
8.2	SISD og SIMD	Langsvar

1 Instruksjonssett

Instruksjon	Opcode	Funct3	Funct7
add	0110011	000	0000000
lw	0000011	010	—
sll	0110011	001	0000000
xor	0110011	100	0000000
andi	0010011	111	—
sw	0100011	010	—
slt	0110011	010	0000000
sra	0110011	101	0100000
sub	0110011	000	0110000
addi	0010011	000	—

I denne oppgaven gir vi 1 poeng for hvert riktige svar. Feil svar/ubesvart gir 0 poeng.

Angi maskinkode for instruksjonen *slt x10, x15, x20*:

- opcode:
- rs1:
- rs2:
- rd:
- func3:
- func7:

Angi maskinkode for instruksjonen *andi x5, x4, 0x7fb*:

- opcode:
- rs1:
- rd:
- imm:

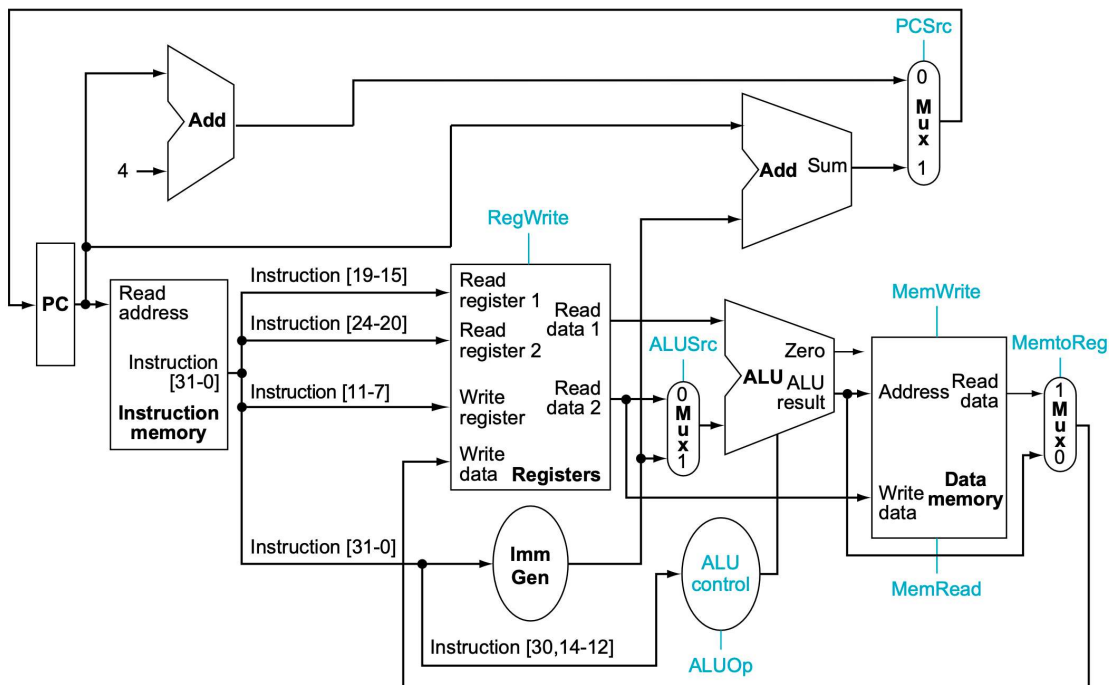
- func3: 111

Angi maskinkode for instruksjonen `sw x30, -16(x14)`:

- opcode: 0100011
- rs1: 01110
- r2: 11110
- imm: 1111 1111 0000
- func3: 010

Maks poeng: 16

2



Denne seksjonen omhandler prosessoren vist over. Du må bruke "don't care" (X) når du kan. I oppgavene som er automatisk rettet, gir vi 0,5 poeng for riktige svar og -0,25 poeng for feil svar. Ubesvarte oppgaver gir 0 poeng.

2.1 Kontrollsignaler 1

Prosessoren skal utføre instruksjonen *slt*. Hvilke verdier skal kontrollsignalene under ha?

RegWrite:

ALUSrc:

PCSrc:

MemWrite:

MemRead:

MemtoReg:

Maks poeng: 3

2.2 Forklare kontrollsignaler 1

Forklar hvordan kontrollsignalene du oppga i oppgave 2.1 gjør at *slt*-instruksjonen utføres korrekt.

Skriv svaret ditt her. Endringer blir lagret automatisk.

SLT er en R-type instruksjon.

RegWrite = 1: Gjør at write enable er høy og når adresse dekode aktiviserer rd, vil dette medføre at vi skriver til denne, med write data.

ALUSrc = 0: Vi har en R type og skal regne med rs2 mux vil da velge å sende denne verdien til ALU

PCsrc= 0: Vi gjør ikke en branch uansett om resultat fra ALU blir null.

MemWrite og Read = 0: Vi vil ikke skrive eller lese fra minnet da dette kan gi undefined behavior, og kan manipulere data vi ikke har tilgang til, eller bruke lenger tid enn vi trenger.

MemtoReg = 0: Vi velger resultat fra ALU

Ord: 111

Maks poeng: 5

2.3 Kontrollsignaler 2

Prosessoren skal utføre instruksjonen *sb*. Hvilke verdier skal kontrollsignalene under ha?

RegWrite:	0
ALUSrc:	1
PCSrc:	0
MemWrite:	1
MemRead:	0
MemtoReg:	x

Maks poeng: 3

2.4 Forklare kontrollsignaler 2

Forklar hvordan kontrollsignalene du oppga i oppgave 2.3 gjør at *sb*-instruksjonen utføres korrekt.

Skriv svaret ditt her. Endringer blir lagret automatisk.

sb da store byte er en S type instruksjon

RegWrite=0: Vi skal ikke skrive tilbake vi setter derav write enable lav sånn at det ikke skrives noe uønsket.

ALUSrc = 1: Vi må regne ut nye adresse med offset derav adderer vi sammen rs1 data og sign extenda immediate.

PCSrc = 0: Samme som over vi skal ikke brance

MemWrite = 1: Vi skal skrive til minne, vi skriver dataen som kommer inn i write data fra rs2 data.

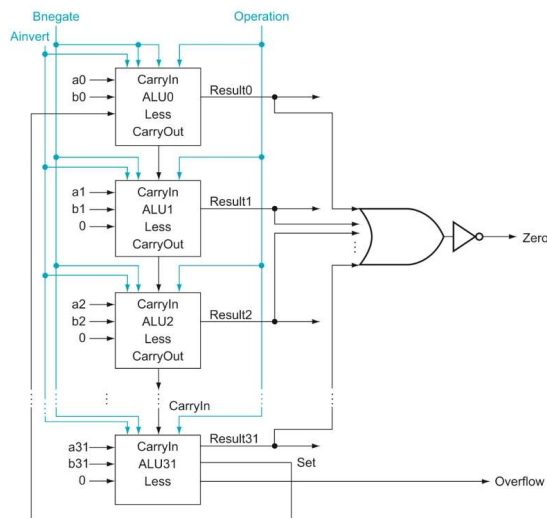
MemRead = 0: Vi skal ikke lese fra minne

MemtoReg = x: vi bryr oss ikke om hva som ligger på write data inn til register da regwrite er 0, og ingenting blir skrevet uansett

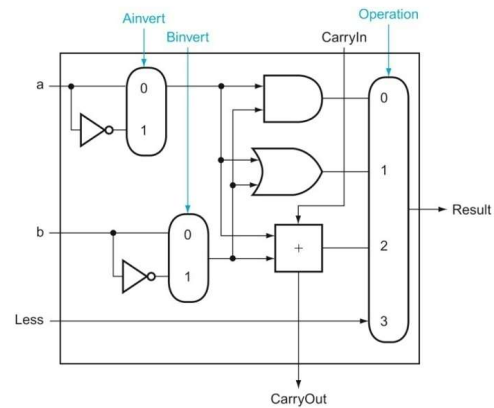
Ord: 114

Maks poeng: 5

3



32-bit ALU



1-bit ALU

Denne oppgaven omhandler den 32-bit brede aritmetisk-logiske enheten (ALUen) vist over (til venstre) som igjen er bygget opp av 1-bit ALUene vist over til høyre. *ALU31* inneholder også logikk for å detektere overflyt, men dette er ikke vist i figuren til høyre.

3.1 ALU kontrollsignaler

ALUen skal utføre operasjonen $a - b$. Angi korrekte kontrollsignaler.

Angi signalverdier i binær med rett antall bit. Vi gir 1 poeng for riktige svar og -0,25 poeng for feil svar. Ubesvarte oppgaver gir 0 poeng.

Bnegate:

Ainvert:

Operation:

Maks poeng: 3

3.2 Forklare ALU kontrollsignaler

Forklar hvordan kontrollsignalene du har oppgitt i oppgave 3.1 fører til at operasjonen $a - b$ utføres.

Skriv svaret ditt her. Endringer blir lagret automatisk.

Vi har en ripple ALU som består av 32 1 bit ALUer og som tar inn sitt tilsvarende a og b signal og carry in og gir ut carry out og tilsvarende result signal. Carry out vil bli sendt til neste ALU i rekka, som tar det inn som carry in signal.

For å gjøre denne spesifikke operasjonen der vi får inn 32 bit data a og b, og skal utføre $a - b$, bruker vi ideen om toerskomplement for å invertre b signalet til sitt negative komplement (kan være både negativt og positivt).

Bnegate = 1 vil gjøre at det inverterte signalet i hver 1 bit ALU er det vi regner med. I den første ALU vil dette signalet også styre en mux som legger til 1.

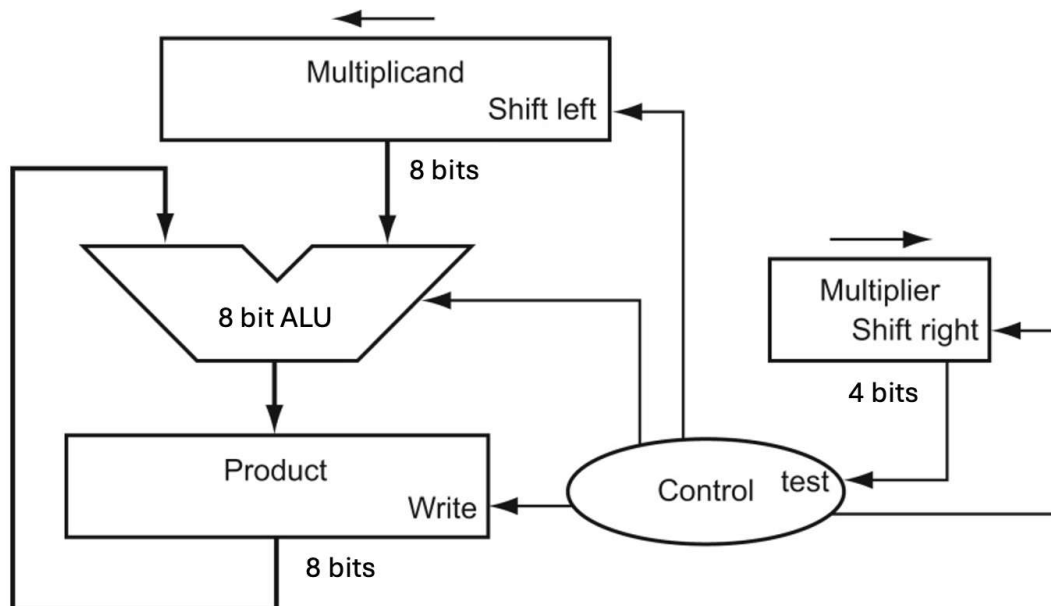
a forblir det samme som den er.

Videre vil operation signalet ta inn hvilken operasjon som skal gjøres inn i mux og velge riktig signal på result. Vi skal gjøre operasjonen addisjon derav velger vi signal 2 som tilsvarer 10 på binær.

Ord: 167

Maks poeng: 4

4



Denne seksjonen omhandler multiplikatoren over.

4.1 Verdier i produktregisteret

Multiplikatoren skal gange de binære tallene 1010 og 1001 med hverandre, altså utføre operasjonen 1010 x 1001. Angi verdien som *Product*-registeret inneholder etter hver iterasjon.

Angi registerverdiene i binær med rett antall bit. Vi gir 1 poeng for hvert riktige svar. Feil svar/ubesvart gir 0 poeng.

Verdien i *Product* etter første iterasjon:

Verdien i *Product* etter andre iterasjon:

Verdien i *Product* etter tredje iterasjon:

Verdien i *Product* etter fjerde iterasjon:

Maks poeng: 4

4.2 Forklare multiplikatoren

Forklar hvordan du kom frem til svaret i oppgave 4.1.

Skriv svaret ditt her. Endringer blir lagret automatisk.

Multiplikasjon i binær er veldig lik den multiplikasjon man lærer på barneskolen. Vi tar først å legger inn multiplikand (1010) slik den er i sitt 8 bit register.

Setter produkt til 0 i sitt 8 bit register og multiplikator (1001) i sitt 4 bits register.

Så ser vi på LSB i multiplikator hvis det er 1 så adderer vi sammen produkt og multiplikand. Så blir multiplikand shiftet mot venstre vi ganger med 2, samtidig som multiplikator blir shiftet mot høyre (her 2 gangeren kommer inn).

Så gjentar vi prosess til vi er ferdig med MSB i multiplikatoren.

Ord: 96

Maks poeng: 4

5

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s		exponent								fraction																					
1 bit		8 bits								23 bits																					

$$(-1)^s \cdot (1 + \text{Fraction}) \cdot 2^{(\text{Exponent} - 127)}$$

Denne seksjonen omhandler flyttall som følger formatet definert over.

5.1 Flyttall til desimaltall

Hvilket desimaltall representerer flyttallet `0x40980000`? Feil svar/ubesvart gir 0 poeng.

Svar:

Maks poeng: 2

5.2 Forklare flyttall

Forklar hvordan du kom frem til svaret i oppgave 5.1

Skriv svaret ditt her. Endringer blir lagret automatisk.

Først skriver vi om til binær fra hexa:

0100 0000 1001 1000 0000 0000 0000 0000

Vi ser at vi har $s = 0$, posetivt tall

$\text{exponent} = 100\ 0000\ 1 = 2^7 + 2^0 = 129 \Rightarrow 129 - 127 = 2 \Rightarrow 2^2 = 4$

$\text{fraction} = 001\ 1000\ 0000\ 0000\ 0000\ 0000 = 2^{-3} + 2^{-4} = 3/16 \Rightarrow$

$1 + 0.1875 = 1.1875$

$1 * 1.1875 * 4 = 4.75$

Ord: 69

Maks poeng: 6

6.1 Samlebånd, kontrollsignaler

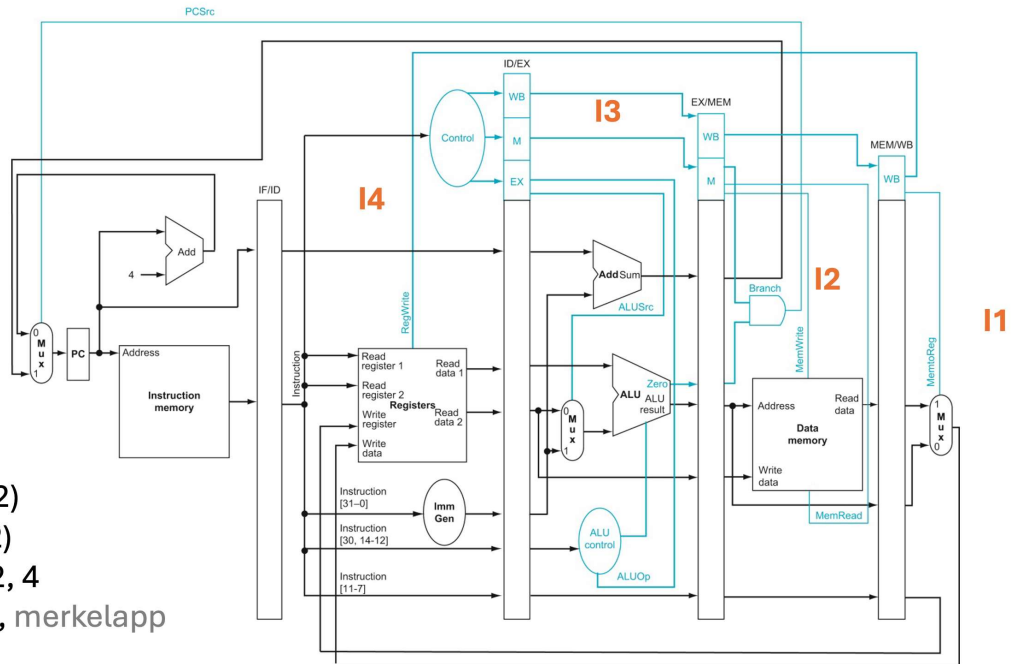
merkelapp:

I1: `sw x1, 0(x2)`

I2: `lw x1, 4(x2)`

I3: `addi x3, x2, 4`

I4: `beq x4, x5, merkelapp`



Figuren over viser en spesifikk klokkesykel i utføringen av et program på en samlebåndsprosessor. Hva er verdiene til følgende kontrollsignaler i denne klokkesykelen?

Du må bruke "don't care" (X) når du kan. Vi gir 0,5 poeng for riktige svar og -0,25 poeng for feil svar. Ubesvarte oppgaver gir 0 poeng.

ID/EX ALUSrc

1

ID/EX MemWrite

0

ID/EX MemRead

0

ID/EX RegWrite

1

EX/MEM MemWrite

0

EX/MEM MemRead

1

EX/MEM RegWrite

1

EX/MEM MemtoReg

1

MEM/WB RegWrite

0

MEM/WB MemtoReg

X

Maks poeng: 5

6.2 Forklare samlebånd

Hvordan sørger kontrollsignalene for at programmet i oppgave 6.1 utføres korrekt? Forklaringen skal dekke alle kontrollsignaler unntatt *ALUOp* for instruksjonene I1, I2, og I3.

Skriv svaret ditt her. Endringer blir lagret automatisk.

I et samlebånd så beveger kontroll signalene seg sammen med instruksjonen gjennom samlebåndet og blir lagret i de midlertidige registrene mellom hvert steg. Når vi nå er i denne klokkesykkelen.

Instruksjon 3 er i execute steget. Kontroll signalene ligger i ID/EX registeret, disse vil nå ligge på Q utgangen til registrene, mens kontrollsignalene til I4 vil etterhvert ligge på D inngangen og vente. I EX steget vil vi bruke kontroll signalene ALUOp og ALUSrc. Videre vil de resterende propagere videre med instruksjonen og brukes i kontroll av senere steg i instruksjon.

ID/EX ALUSrc 1: Gjør at vi velger sign extenda immediate og regner ut en add med denne og rs1.

ID/EX MemWrite/ MemRead =0: Disse brukes i mem steget og viser at vi ikke skal gjøre noe med minnet

ID/EX RegWrite =1: I WB steget skal vi skrive tilbake verdi fra ALU til rd

Instruksjon 2 er i mem steget og er en lw, i denne sykkelen bruker den memwrite og memread, mens de andre brukes i WB steg

EX/MEM MemWrite = 0: vi skriver ikke til minnet

EX/MEM MemRead = 1: vi leser fra minnet fra den gitte adressen regnet ut i EX

EX/MEM RegWrite = 1: senere skal vi skrive til registersfila det vi leste fra minnet

EX/MEM MemtoReg = 1: Vi skriver tilbake det vi hentet fra minnet

I WB steget har vi instruksjon 1 som er en sw

MEM/WB RegWrite = 0: Skal ikke skrive til registersfila

MEM/WB MemtoReg =X: det har ikke noe å si hvilken verdi som ligger å venter på write data da denne ikke skrives uansett.

Det står i oppgave at alle kontrollsignaler skal beskrives, hvis dette er ment å ta med alle for alle instruksjoner , så kommer det her, hvis ikke se bort fra dette:

Branch/PCsrc brukes for å finne ny adresse ved et hopp: Siden vi i i instruksjon 1, 2 og 3 ikke skal gjøre noe hopp selvom resultat i ALU blir null settes denne til 0 i de tilsvarende. For I4 settes denne til 1, sammen med ALUOp til sub for å styre PCsrc signal som styrer neste PC verdi.

For I4 settes ALUSrc til 0 for å sammenligne rs1 og rs2. Memwrite/read samt regwrite til 0 ikke noe skal gjøres med minnet, og ikke noe skal skrives tilbake. MemtoReg blir x.

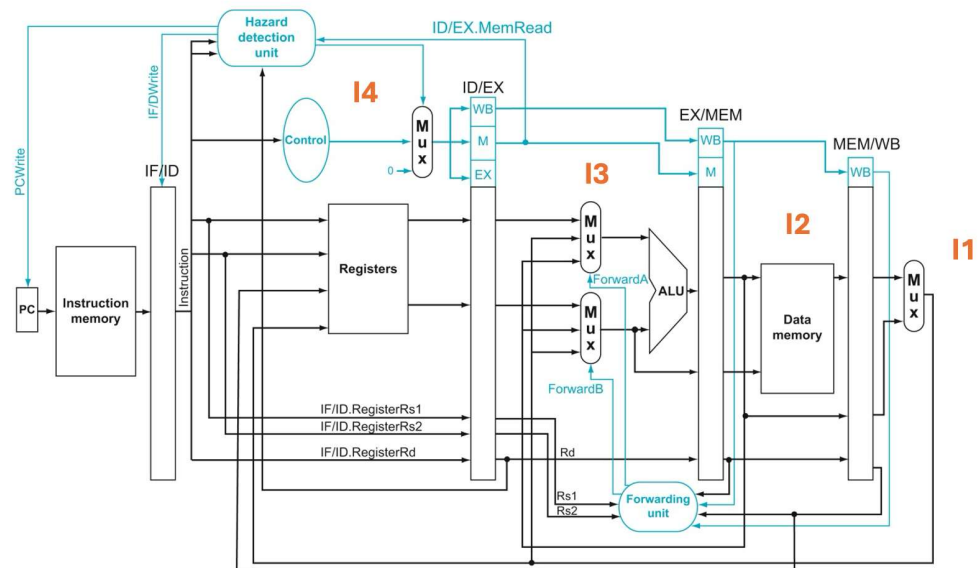
For I1 vil vi gjøre en sw og alusrc er 0 vi velger å regne med immediate for å finne ny adresse. Memwrite = 1 vi skriver til minnet og memread = 0.

For I2 vi alusrc være 1 for å regne ny adresse.

Maks poeng: 6

6.3 Korrekt utføring

I1: add x5, x1, x2
 I2: add x6, x5, x2
 I3: add x7, x5, x6
 I4: add x8, x7, x2



Figuren over viser en ny klokkesyklus for et annet program på en annen prosessor. Hvordan sørger prosessoren for at instruksjon I3 utføres korrekt?

Skriv svaret ditt her. Endringer blir lagret automatisk.

I3 er en add instruksjon som bruker x5 og x6 sin data, men vi ser at disse verdiene blir regnet ut i I1 og I2, noe som gjør at de på denne 5 stegs samlebånd ikke er skrevet tilbake til registerfila i denne syklens, og vi risikerer uten fare løsning å bruke feil data.

Dette løses ved at vi bruker en forwaring unit, denne funker ved at den tar inn rs1 og rs2 fra I3 videre tar den inn rd fra EX/MEM og MEM/WB. Den regner ut om noen av disse er like, her finner den at rd i EX/MEM altså for instruksjon2 er lik rs2, den sjekker også om RegWrite er 1, den finner ut at det er den og da sender den inn til mux på ForwardB signalet 01 og mux velger da å sende inn verdien fra EX/MEM som da er verdien x6 regnet ut inn til ALU noe som ordner opp i datafaren. Tilsvarende for x5 som ligger i WB steget.

Ord: 165

Maks poeng: 5

7.1 Adresseformat

Anta et 2-veis settassosiativt hurtigbuffer med en kapasitet på 256 KiB og en blokkstørrelse på 128 byte. Minneadressene er 32 bit lange, og minnet er byte-adresserbart. Hvilke bit i adressen brukes til offset, indeks, og tag?

Bitposisjonen til den minst signifikante biten er 0. Vi gir 0,5 poeng for rett svar og 0 poeng for feil svar/ubesvart.

	Minst signifikante bit	Mest signifikante bit
Offset	0	6
Indeks	7	16
Tag	17	31

Maks poeng: 3

7.2 Forklare adresseformat

Forklar hvordan du kom frem til adresseformatet i oppgave 7.1.

Skriv svaret ditt her. Endringer blir lagret automatisk.

Vi vet at 2-veis settassosiativt medfører at vi har 2 blokker/set

Vi har 128 Byte/ Blokk

$256 \text{ KiB} = 256 * 2^{10} \text{ Byte}$

For å regne Offset, vi vet at vi har byte adresserbart minnet derav tar vi $\log_2(128) = 7$. Altså for å adressere hver Byte i en blokk må vi ha 7 bit offset=> 0, 6

For å regne ut index må vi finne antall set: Da tar vi $256 \text{ KiB} / (128 * 2 \text{ Byte/ Set}) \Rightarrow 1 \text{ KiB set} \Rightarrow 10 \text{ bit}$ for å representere alle Byte i index. 7,16

Resten av adressen ligger i TAG som da er 17,31. Dette tilsvarer 15 bit som blir riktig i en 32 bit adresse.

Ord: 112

Maks poeng: 6

7.3 Aksestid

Enhet	Latens (klokkesykler)	Bomrate
Nivå 1 hurtigbuffer	3	15%
Nivå 2 hurtigbuffer	10	30%
Minnet	75	-

Hva blir gjennomsnittlig minneaksestid for systemet beskrevet i tabellen over?

Svar:

Maks poeng: 2

7.4 Forklare aksestid

Forklar hvordan du kom frem til svaret i oppgave 7.3.

Skriv svaret ditt her. Endringer blir lagret automatisk.

For å regne ut AMAT bruker vi den rekursive formelen vi vet at 0.85 av gangene treffer vi L1 derav $0.85 \cdot 3$

Vi bommer 0.15 for L1, vi må da regne ut AMAT for å hente ting fra L2.

I L2 treffer vi 0.7 og vi bruker her latens fra dette nivået og det vi brukte på å lete og komme oss til L1 = $10 + 3 = 13 \Rightarrow 13 \cdot 0.7 = 9.1$

I L2 bommer vi 0.3 ganger her må vi summere opp latens fra L1, L2 og hovedminnet = $3 + 10 + 75 = 88$ sykler.

$88 \cdot 0.3 = 26.4$

AMAT L2 = $26.4 + 9.1 = 35.5$

Samlet $0.85 \cdot 3 + 0.15 \cdot (35.5) = 7.875$

Ord: 123

Maks poeng: 4

7.5 TLB

Hva gjør et "Translation Lookaside Buffer (TLB)", og hvorfor er de viktige i datamaskiner som støtter virtuelt minne?

Skriv svaret ditt her. Endringer blir lagret automatisk.

I dagens maskiner utnytter vi Virtuelt minne. Dette er en strategi der vi har fysiske minne adresser som blir kontrollert av OS, og virtuelle minne adresser for prosessene som kjøres på maskinen. For å oversette de virtuelle adressene til en prosess over til fysiske minneadresser som kan aksessere har vi en sidetabell styrt av OS som ligger i hovedminnet. Det tar derimot lang tid å aksessere hoved minnet, noe som er begrensende for ytelsen til maskinen, derfor ønsker vi å inføre et hurtigbuffer med oversettelsene for en prosess, det er her TLB kommer inn.

TLB:

Hver prosess har hver sin TLB der vi henter inn oversettelsene fra virtuelle adresser til fysiske adresser for prosessen. Dette hurtigbuffer er full asosiativt noe som betyr at vi ikke har noe index og da ikke trenger å kaste ut pages som deler index, dette øker treffraten da vi bare kaster ut en page hvis det er "lenge" siden vi har brukt den. I det som vi ser på data i vanlig hurtigbuffer ligger page oversettelsen som da er de mest signifikante bitsa til en page. Vi ønsker å ha relativt store pages (4KiB)(debaterbart), sånn at når vi har hentet inn en page for noe data så utnytter vi lokalitet i rom ved at vi trolig har lastet inn pages for annen data i denne prosessen. Dette gjør at treffrate i TLB ved å utnytte fullasosiativitet(lokalitet i tid) og store pages(lokalitet i rom) gjør at vi veldig ofte treffer i TLB og vi da minimerer den negative ytelses-påvirkningen som kommer av å ha virtuelle adresser. Vi har nå en rask aksessering og vi er forsikret om at de ulike prosessene ikke leser og skriver til hverandres minne.

Ord: 282

Maks poeng: 4

8.1 Register renaming

Hva gjør "register renaming" og hvorfor brukes denne teknikken i prosessorer med høy ytelse?

Skriv svaret ditt her. Endringer blir lagret automatisk.

Man bruker register renaming i høy ytelse prosessorer for å unngå WAW og WAR farer i en out of order samleband. I høy ytelses samleband vi har sett på i dette faget ligger denne som en del eller da etter ID steget og før dispatch steget. De register adressene som skal manipuleres i samlebandet får et fysisk register som de ligger på under utførelsen i issue buffer, dette gjør at i en høy ytelse der vi gjør ting out of order vil se at si hvis vi først gjør en divisjon som tar 20 klokkesykler I1, også en addisjon som 3 klokkesykler (vi regner med flytall) I2, så vil det at vi har ulike fysiske registre gjøre at vi motiviker race conditions der prosessoren vil være ferdig med I2 før I1. Da vil I2 selvom den er ferdig først ligge senere i ROB, og må altså vente på å skrive resultatet tilbake til etter I1.

Ord: 152

Maks poeng: 4

8.2 SISD og SIMD

Hva er den viktigste forskjellen på en SISD og en SIMD maskin? Nevn en fordel og en utfordring med SIMD-maskiner.

Skriv svaret ditt her. Endringer blir lagret automatisk.

Dette går på Flynns taksonomi og går ut på hvordan vi klassifiserer prosesserings enheter/maskiner. En SISD enhet vil jobbe på et dataelement og en instruksjon i hvert steg, dette er de klassiske 5 stegssamlebåndene våre. I en SIMD maskin vil vi regne på flere dataelementer for en instruksjon i stegene våre, et eksempel er en SM/GPU kjerne i en GPU prosessor. Den vil jobbe på flere data/ data tråder med en tilhørende kernel/instruksjonstråd, en fordel med denne og generelt SIMD maskiner er at de kan jobbe med parallele data feks når vi skal manipulere video der vi gjør samme instruksjon på mange uavhengige dataelementer. Vi får da en høyere troghput/gjennomstrømning. En utfordring med en slik maskin er hvis vi har iregulær kode/kritiske seksjoner der data er avhengig i prosessen vår, kan vi ikke dele opp data i parallell. Da kan vi plutselig bare regne på et dataelement av gangen og den ekstra hardware i SIMD(som kanskje er spesialisert på en oppgave med mange dataelementer), som ofte går utover evnen til å gjøre generelle oppgaver, den mister vi.

Ord: 178

Maks poeng: 6